CHAPTER 6
SYSTEM ANALYSIS AND SYSTEM DESIGN

6.1 System Analysis and Design

6.2 Systems Development Life Cycle (SDLC)

6.2.1 Needs of SDLC

6.2.2 SDLC Cycle

6.3 Spiral Model

6.4 Class Diagram

6.1 System Analysis and Design

System Analysis and Design (SAD) is the backbone of software development, guiding the creation of efficient and effective systems to address organizational needs. At its core, SAD is about understanding the current state of affairs within an organization, envisioning a better future, and architecting the pathway to get there.

During the analysis phase, SAD professionals delve deep into the existing systems and processes, meticulously identifying strengths, weaknesses, and opportunities for improvement. Through interviews, surveys, and observation, they gather requirements from stakeholders, ensuring that every voice is heard and every need is accounted for.

With requirements in hand, the design phase begins, where SAD experts transform abstract ideas into concrete plans. They define the system architecture, user interfaces, and data structures, ensuring that the design aligns closely with the gathered requirements and the overarching goals of the organization.

Implementation brings the design to life, as developers translate the blueprints into functional code. Rigorous testing follows, with each component scrutinized to ensure its reliability, security, and performance. User acceptance testing ensures that the system not only meets technical specifications but also satisfies the needs and expectations of its intended users.

Deployment marks the culmination of the SAD process, as the new system is introduced into the organization's ecosystem. With proper training and support, users adapt to the new system, embracing its capabilities and maximizing its potential to drive efficiency and innovation.

But the journey doesn't end there. Maintenance and support are ongoing responsibilities, as SAD professionals monitor the system, address issues, and incorporate updates to keep it running smoothly and meeting the evolving needs of the organization.

Throughout every phase of System Analysis and Design, collaboration, communication, and a relentless focus on the end-users are paramount. By aligning technology with business

objectives and user needs, SAD professionals play a vital role in shaping the future success of organizations across industries.

6.2 Systems Development Life Cycle (SDLC)

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement. Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

6.2.1 Needs of SDLC

The development team must determine a suitable life cycle model for a particular plan and then observe to it.

Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner. When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure. This problem can be defined by using an example. Suppose a software development issue is divided into various parts and the parts are assigned to the team members. From then on, suppose the team representative is allowed the freedom to develop the roles assigned to them in whatever way they like. It is possible that one representative might start writing the code for his part, another might choose to prepare the test documents first, and some other engineer might begin with the design phase of the roles assigned to him. This would be one of the perfect methods for project failure.

A software life cycle model describes entry and exit criteria for each phase. A phase can begin only if its stage-entry criteria have been fulfilled. So without a software life cycle model, the entry and exit criteria for a stage cannot be recognized. Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.

6.2.2 SDLC Cycle

SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:

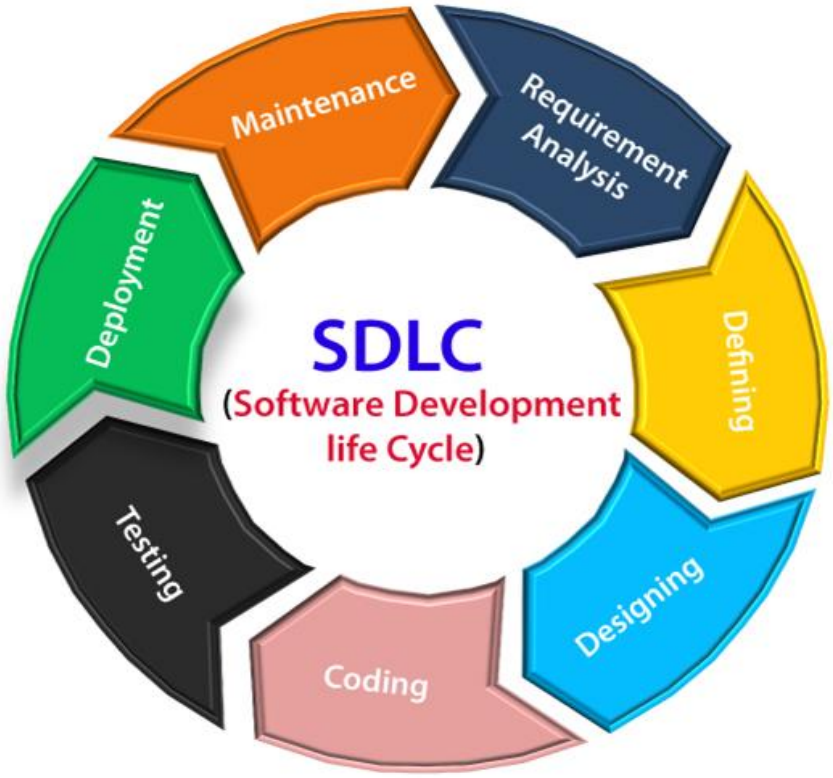

Figure 6.2.1: SDCL Cycle

- The stages of SDLC are as follows:

Stage1: Planning and requirement analysis

- Requirement Analysis is the most important and necessary stage in SDLC.

- The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.
- Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Stage2: Defining Requirements

- Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.
- This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software

- The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project

- In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code.
- Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

- After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.
- During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment

- Once the software is certified, and no bugs or errors are stated, then it is deployed.
- Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment. After the software is deployed, then its maintenance begins.

Stage7: Maintenance

- Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time. This procedure where the care is taken for the developed product is known as maintenance.

6.3 Spiral Model

- The Spiral Model is a Software Development Life Cycle (SDLC) model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a phase of the software development process.
- The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
- As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
- It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.
- The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process. It consists of the following phases:
- Planning

  o The first phase of the Spiral Model is the planning phase, where the scope of the project is determined and a plan is created for the next iteration of the spiral.

- Risk Analysis
  o In the risk analysis phase, the risks associated with the project are identified and evaluated.
- Engineering
  o In the engineering phase, the software is developed based on the requirements gathered in the previous iteration.

- Evaluation
  o In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.

- Planning
  o The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.
- The Spiral Model is often used for complex and large software development projects, as it allows for a more flexible and adaptable approach to software development. It is also well-suited to projects with significant uncertainty or high levels of risk.
- The Radius of the spiral at any point represents the expenses(cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.
- Each phase of the Spiral Model is divided into four quadrants as shown in the above figure. The functions of these four quadrants are discussed below:
- Objectives determination and identify alternative solutions:
  o Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.
- Identify and resolve Risks:
  o During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.
- Develop the next version of the Product:

o During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.

- Review and plan for the next Phase:
  o In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

6.4 Class Diagram

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also construct executable software code.
It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development. Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.

- **Purpose of Class Diagrams**
  o The main purpose of class diagrams is to build a static view of an application. It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is one of the most popular UML diagrams. Following are the purpose of class diagrams given below:
    - It analyses and designs a static view of an application.
    - It describes the major responsibilities of a system.
    - It is a base for component and deployment diagrams.
    - It incorporates forward and reverse engineering.

- **Benefits of Class Diagrams**
  o It can represent the object model for complex systems.
  o It reduces the maintenance time by providing an overview of how an application is

structured before coding.

o It provides a general schematic of an application for better understanding.

o It represents a detailed chart by highlighting the desired code, which is to be programmed.

o It is helpful for the stakeholders and the developers.